

Sight Reading Pro

Final Project Report - 6298/10500 words

Theresa Thoraldson

2026-09-28

Abstract

This report describes the motivation, literature review, design, implementation, and evaluation of Sight Reading Pro, a full-stack machine learning application for sight-reading practice. The template for this project is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal. This project code be found here: <https://github.com/tthoraldson/uol-final-project/>

Contents

Introduction - (720/1000 words)	3
An overview	3
Why Would People Want This Project?	3
Existing/Similar Projects	4
Literature Review (1903/2500 words)	6
Exploring Sight-Reading in Education	6
Pitch Estimation, Tempo Estimation, Audio Evaluation Models	6
Music Generation	7
Other Models	9
Libraries	10
The Research Gaps	11
Design - (926/2000 words)	12
Domain and Project Users	12
Design Overview	12
Design Choices	19
Implementation (1653/2500 words)	21
Docker	21
Docker Compose	21
Frontend	23
Main Backend API	24

Model APIs	25
At a high level	27
Evaluation (721/2500 words)	27
Model Evaluation	27
Failed Approaches	31
Limitations	32
Conclusion (375/1000 words)	32
If I were to do it again...	32
Further Work	33
Acknowledgements	35
References	35

Introduction - (720/1000 words)

The template I have chosen is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal.

In music, there is a concept called sight-reading. It involves being given a piece of sheet music and trying to play or sing it right away with little or no preparation. According to the National Association for Music Education, the ability to sight-read music has many benefits including increased confidence, stronger foundations in rhythm and pitch, and less stress when it comes to learning new music pieces [7].

An overview

“Sight Reading Pro”, is an application where users can practice and master sight-reading. Users can use any instrument of their choosing, including voice, to practice their sight-reading abilities.

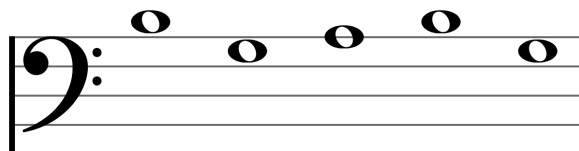


Figure 1: Example sight-reading passage

Sight reading pro is a web application along with multiple APIs, utilizing various state of the art machine learning models to generate music passages for users to practice their sight-reading abilities. The user gets

Users can customize the complexity and difficulty of the generated passages using settings. Users can also select between different styles of passages, such as classical or jazz.

Why Would People Want This Project?

Personally, I’ve always wanted to get better at sight-reading but haven’t found any existing products on the market that make me stick with working through exercises. The generated exercises are often boring, and only target one music area at a time (i.e. advance rhythms, chords, harmony, etc). Having the ability to customize the exercises, and adjusting them to meet my current music learning goals would be amazing.

The existing free projects on the market have limited instrument support, and have no way to track exercise history. The paid options are more focused on music education, instead of individual learning goals. The paid options are also

out of reach to users that want to learn sight-reading, but don't have the budget to do so.

My goals for the project are to make a free, open source, individualized sight-reading platform that can work with any instrument.

Existing/Similar Projects

Sight Reading Factory

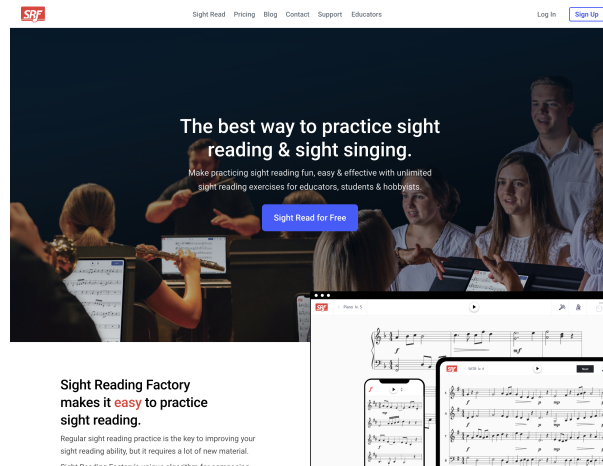


Figure 2: Sight Reading Factory screen capture

Sight Reading Factory[26] is a platform for sight-reading exercises mainly aimed towards educators. This platform supports ~30 instruments, and offers different difficulties of music passages to play. They offer generated music exercises for learners to practice with, getting feedback with each attempt at playing a passage.

Advantages Sight Reading factory has a wide variety of features, including the ability to assess how well a user did on a given passage of music, and the ability to control how difficult generated passages are by limiting the note range, tempo, and rhythmic complexity. Sight Reading Factory supports both general audio/microphone input and MIDI.

Sight reading factory allows you to keep track of what passages you have generated, all of the sight-reading attempts you have made on a given passage, and it offers assessments on new blind passages. This product is the most robust with features that I have found.

Disadvantages I believe the biggest disadvantage to Sight Reading Factory is the fact that it's a subscription, and starts at \$45 per user per year. There's a

free trial, but you have to enter credit card information in order to try it.

Sight Reading factory supports 30 instruments, but has no generic option if your instrument is not supported. You must log in to do anything, which can be just enough friction for learners that want to try sight-reading without committing to a product. This product is also heavily influenced by music educators, and a lot of the features are what you would expect to be in a classroom environment. For example, getting assigned passages to work on.

sightreading.training

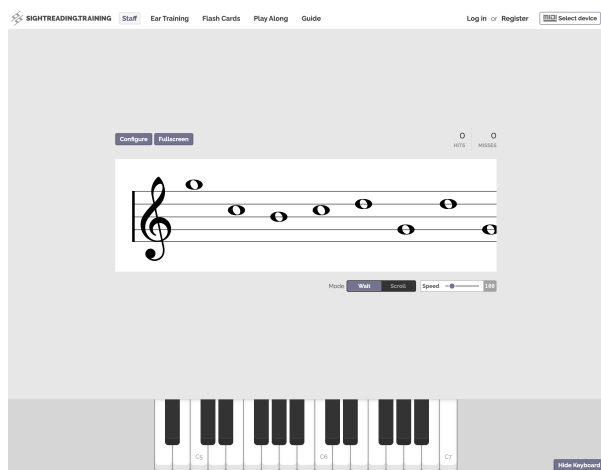


Figure 3: sightreading.training screen capture

sightreading.training[27] is a free and open source sight-reading platform. It offers sight-reading tools, and other tools for learning and playing music.

Advantages sightreading.training supports MIDI, or allows the user to play on a piano that's in the browser. Having a virtual piano is a cool idea, as it makes sight-reading accessible to everyone, including those that don't have an instrument of their own. This website is also free, and you can get started practicing right away. Users have the ability to login and save settings for their sight-reading passages, but not the passages themselves.

Disadvantages sightreading.training only supports MIDI and the virtual piano. If you have an instrument that you can't plug into your computer, you can't use this application. There are no settings to change rhythmic complexity. All of the notes have the same length.

Literature Review (1903/2500 words)

Exploring Sight-Reading in Education

Understanding sight-reading, and how to improve this ability has been an area of study for over 100 years. One research study shows that aural-spatial skills (ear training) and technical proficiency skills were both essential to sight-reading[45].

In more recent times, a review was done on AI-education research related to music education. It concludes that “AI is under-utilized for generation despite its potential for education”[40]. The music generation part of the literature review goes deeper into music generation related to music exercises for general technical improvement.

Pitch Estimation, Tempo Estimation, Audio Evaluation Models

There are lots of existing pre-trained models available that do different tasks related to music. This is an overview of the models I found most relevant to this project, including the models that I ended up using.

CREPE

CREPE is a deep convolutional neural network that does pitch estimation [46]. Given an audio file, such as a .wav or .mp3 file, CREPE will give a predicted frequency in hertz alongside a confidence score for every 10 milliseconds. Having a confidence interval allows for

There is also a demo of CREPE[3] that runs fully in the browser using Tensorflow JS (tfjs)[30]. It shows real time pitch estimation based on microphone input. In regards to this project, being able to have real time pitch estimation would allow for immediate feedback on a music passage.

TempoCNN

TempoCNN[53] is a group of Convolutional Neural Network (CNN) models that estimate a given audio file’s tempo, measured in beats per minute (BPM). These models were trained on datasets that included mainly ballroom dancing music, and electronic dance music. This paper does note that there is a lack of various genres of music, including jazz, classical or reggae, and believes that the models would perform better if they had found and included this kind of data.

For my project I believe TempoCNN is a good candidate model for being able to automatically detect the tempo that the user’s recording was played at, and also use it for giving feedback on a given passage.

Music Generation

Basic Pitch

Basic Pitch[39] is a model built by spotify that takes in an audio file (.wav or .mp3), and returns a MIDI file containing either the single pitch (one instrument/voice) or multi pitch (multiple instruments/voices/notes). Basic Pitch works in the realm of “Automatic Music Transcription”, an area of research used to automatically create symbolic representations of music.

I believe Basic Pitch is a good candidate model that I can use to create backing tracks, and be able to have them stored as midi files. This way the project can take advantage of technologies such as WebMIDI[34] for playback of a generated passage.

MusicGEN

MusicGEN[41] is capable of taking a text based input, and generating a full song. This model can also be prompted with an existing melody and a text prompt, and output a song based on the original melody.

This series of models is promising for offering customized backing tracks for generated sight-reading exercises. The exercises could be tailored to include the correct genre of backing track.

While the music generated is really high quality, the models are also large and require a lot of compute. The smallest model in the MusicGEN model series contains 300 million parameters, and requires ~16gb of GPU RAM to run.

NotaGen

NotaGen[57] is a model that generates classical sheet music. The paper proposes a new “CLaMP-DPO” method for reinforcement learning, which increases musicality when compared to traditional human annotation or predefined rewards. CLaMP 2[61] is a multi-modal music information retriever that was trained across 101 languages. NotaGen uses CLaMP 2 in its reinforcement learning CLaMPD-PO, where it takes the CLaMP model to do iterative optimization, and then DPO[48].

This model requires at least 8GB of GPU RAM to run the smallest model. One goal of “Sight Reader Pro” is to have low response times for any generative process. Because of the large compute requirements for NotaGen, I don’t believe it will be a good fit for the project.

Text To Music

text-to-music[24] is a model that takes in a prompt, and produces ABC notation[1] from that prompt. It’s a BART[48] based model that has been finetuned with a dataset similar to wikimusictext[25]. I find this to be one of the more compelling models, as the prompt examples contain the same kind of natural language that I’m looking to use in Sight Reader Pro:

Example training prompts for text-to-music, directly from model card on hugging face:

- This is a traditional Irish dance music. Note Length-1/8
Meter-6/8 Key-D
- This is a jazz-swing lead sheet with chord and vocal.

I find the ability to use natural language to prompt for specific time signatures and music styles to fit perfectly with that I'm going to build with Sight Reader Pro. I also like that the output is in ABC format, which is just plain text. This makes it easy to store, and easy to process.

The training dataset used also contained the genre of music for each tune it was trained on, which is an important feature to have for when I generate prompts for creating new music.

Chat Musician

Chat Musician[63] is a 4 billion parameter model based on LLAMA2[56] that takes natural language prompts, and can do three tasks: Music Theory QA, Composing (using ABC notation[1]), and chatting. They created their own dataset, which they call *Music Pile*, that includes a variety of information outside of music including Wikipedia[37], Irishman[59], and synthetic music data that they created using Chat-GPT 4[51]. The model performs extremely well on music understanding, music reasoning and music knowledge tasks. The paper talks about music generation as a way to compress music data into the ABC format for better compression, but doesn't talk about music generation as it's own goal.

Text2Midi

Text2Midi[31] is an LLM that generates midi files from text descriptions. It takes LLM Embeddings from the text description, then uses a Transformer decoder to generate tokens that are then converted to a midi file.

MIDI[11] is a standard interface to define communication between instruments and a computer. MIDI can be live, like using a keyboard as a MIDI input, or MIDI can be a file format that's used to define the pitch, rhythm and tone of an instrument or an ensemble.

Using MIDI as a way to generate music seems like an interesting approach. There seems to be a handful of libraries that can handle MIDI to ABC notation and vice versa, but that doesn't seem to be the norm. I still want to experiment with generative MIDI models, as they have the potential to generate different music from the text to ABC style models listed above.

MidiLLM

Similar to Text2Midi, MidiLLM[60] is an LLM that produces MIDI. Text2Midi is based on Llama 3[44]. It uses a two stage training process so that the model can understand text related to music, and also MIDI. MidiLLM outperforms Text2Midi on the TheoryTab dataset[42], which specializes in multitrack music generation. This model is compatible with the HuggingFace Transformers library[58].

Considering my application is for sight-reading, which is a solo task, I don't believe using a model that specializes in creating MIDI for ensembles .

AudioGen

AudioGen[47] is an auto-regressive generative model that creates audio samples based on text prompts. It was trained on a variety of different . The public model found on HuggingFace[8] has 1.5 billion parameters.

One of my original ideas for Sight Reader Pro was to have the ability to create backing tracks for the generated sight-reading exercises. While I could also generate midi for this task, having more realistic audio that sounds like a real guitar, or a reach orchestra seems appealing.

My main worry with this model, similar to MusicGen[41], is that it's too large, and too computationally expensive to use alongside simple, ABC notation based sight reading exercises. It would be cool to incorporate something like this into a base set of exercises on Sight Reading Pro in the future, where the ABC and backing track are already generated, but this will not work for on the fly sight-reading exercise generation.

Other Models

Qwen, Qwen ONNX

Qwen3[62] is an open source LLM, and is a popular alternative to both Chat GPT and Claude. It can

On HuggingFace, there's an ONNX Runtime[16] implementation of Qwen3[17], having 0.6 billion parameters. ONNX Runtime can run models both on mobile devices and directly in the browser. Having a cross-platform compatible model for any conversation or retrieval tasks could be nice! And using something like TensorflowJS[30] or TransformersJS[10] to run Qwen 3 directly in the browser is appealing.

I believe this would be more of a stretch feature than anything I would include in this iteration of Sight Reader Pro.

Llama

Llama 2[56] and Llama 3[44] are a series of LLMs put out by Meta. As listed above, they're commonly used as base models for building fine tuned and task specific models. The Llama series is great at typical LLM tasks like conversation and retrieval (Just like Qwen!).

Similar to Qwen 3, using a Llama model directly is a stretch feature.

Libraries

Librosa

Librosa[50] is a python library that has lots of audio processing functions. It offers pre-made models for tasks like onset detection, rhythm features, pitch and tuning, onset detection, and much more.

I plan on using librosa to validate the results of some of the larger models I'll be using. Librosa can handle beat tracking and tempo related tasks, and it also has an implementation of Pyin[49], a frequency estimator does the same task as CREPE[4], as mentioned above.

music21

"music21[36] is a Python-based toolkit for computer-aided musicology." It has tons of helper methods to convert between common types (abc -> midi), It has has a large corpus of various free music in different formats for testing. I believe this library will be helpful for quickly converting between music formats, and maybe finding some testing data.

muspy

muspy[[43]][14] is library mainly focused on creating music analysis related pipelines, and facilitating music i/o between other popular music formats and libraries. It supports both ABC notation[1] and MIDI, along with many more. It also has it's own set of muspy specific classes, focused on symbolic music representation.

This library will be helpful if I want to explore storing the sight-reading exercises in a more symbolic way, and it could also be helpful for it's music i/o functions

mir_eval

mir_eval[[52]][12] describes itself as " Python library which provides a transparent, standardized, and straightforward way to evaluate Music Information Retrieval systems." It has tons of methods specifically for Music Information Retrieval, including tempo validation, key detection, and segmentation.

This library will be extremely useful in helping to evaluate how well my selected models perform, and to calculate metrics for users sight-reading exercise attempts.

The Research Gaps

While there are many *text-to-music* machine learning models that satisfy generating new sight-reading exercises, there is a lack of applications that put it to use in real time. There is also a lack of using the AMT style tools and models to produce feedback on sight reading exercises. The two main solutions (Sight Reading Factory[26] and sightreading.training[27]) don't have the ability to customize exercises based on genre, or any prompt for that matter.

Design - (926/2000 words)

As stated in the intro, the template I am choosing is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal.

Domain and Project Users

The goal of this project was to create a sight-reading web application that utilizes various machine learning models to create unique sight-reading exercises. The *target user* for this project is an independent musician that's looking to improve their sight-reading abilities, and is interested in a customized experience based on their instrument of choice, skill level and music genre of choice. The *domain* of this project could be considered to be music education, but I'm aiming for it to be specifically independent music education.

This project is not ideal for users just learning an instrument, as this application requires at least an elementary understanding of reading sheet music, and playing their instrument of choice.

In my initial feedback survey on the mockups, there was almost an even split of people who were musicians, and people who were not. I believe this is the best kind of feedback, as I can iterate on my features to work for everyone who may want to try sight-reading, not just the experts.

Would you describe yourself as a musician, either professionally or as a hobby?

7 responses

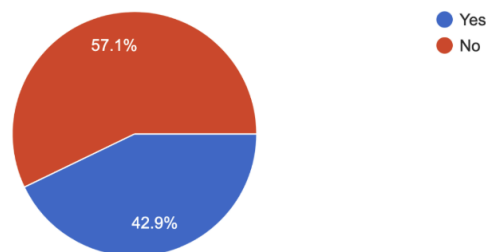


Figure 4: Initial Design Survey: Musician Question

I also asked if users were familiar with the concept of sight-reading. Surprisingly, the majority did! I believe this validates the name, *Sight Reader Pro*, as something that could be recognizable.

Design Overview

The designs below are both my initial mockups for what Sight Reader Pro might look like, and the final version.

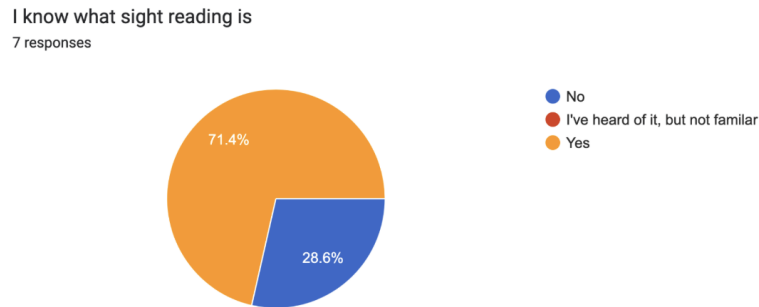


Figure 5: Initial Design Survey: Sight Reading Question

Home Page

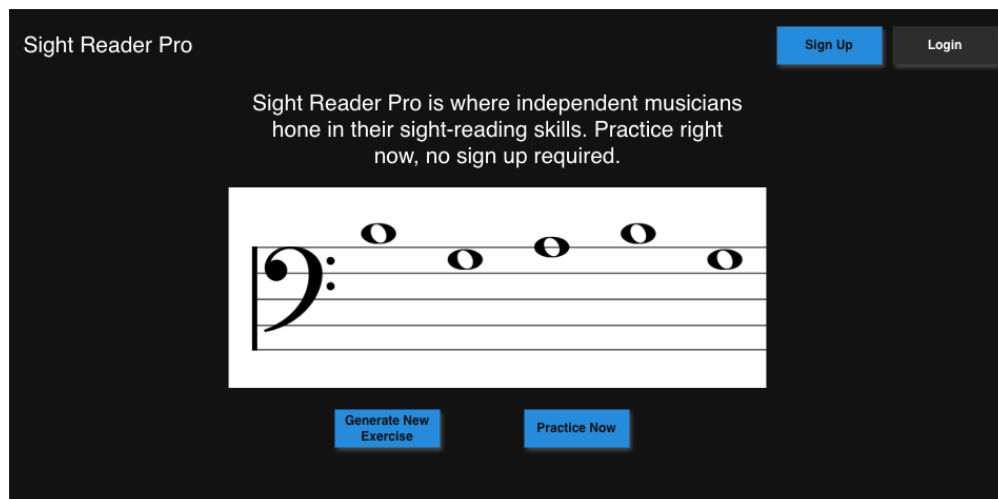


Figure 6: Sight Reading Pro Home Initial Design Mockup

In the initial mockup phase, I was planning on implementing a home page that had the majority of the functionality of the exercise page, but with some marketing flare for first time visitors. After I got into implementing the app, I decided to ax creating a different home page. The exercise page is the only page, and all of the functionality of the final Sight Reading Pro happens there.

Before Recording Exercise Attempt

The mockup exercise page before an attempt is very clean, only including two options: connect to an audio source and record an attempt. If the user selects the record, the record button will switch to a stop button. After the stop button is

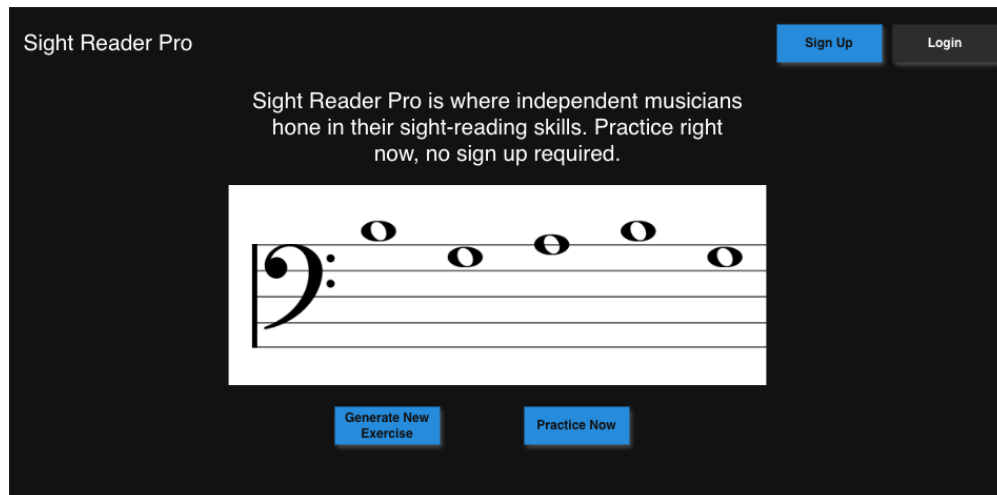


Figure 7: Sight Reading Pro: Home Initial Design Mockup

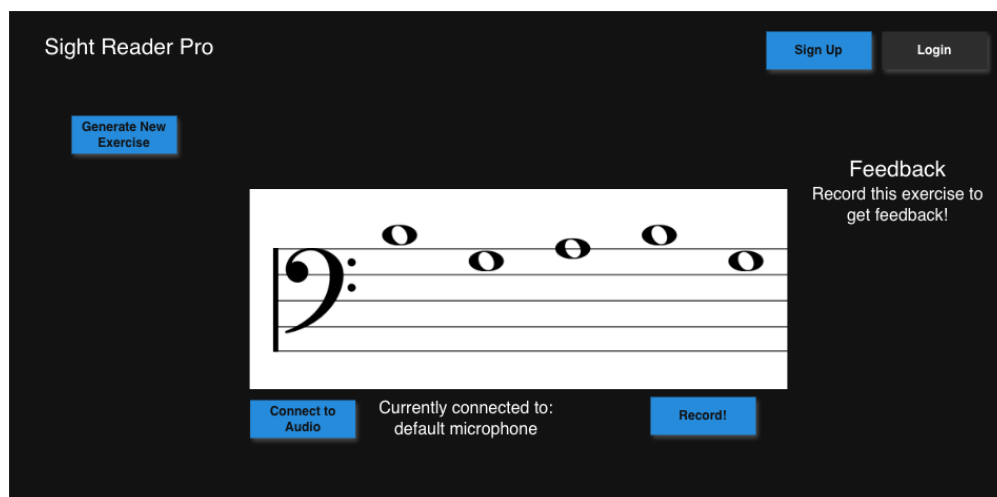


Figure 8: Sight Reading Pro Exercise Before Attempt

selected, the screen will update to the “after recording exercise attempt” screen. In this mockup, there was no option to upload audio.

This is the main exercise page in the application Sight Reading Pro. It's where musicians record their sight reading attempts. Are the options of this page easy to understand?

7 responses

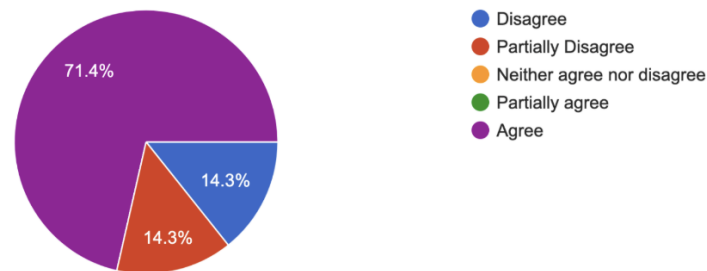


Figure 9: Initial Design Survey: Exercise Before Attempt Feedback

User Feedback The majority of the users surveyed agreed that the options of this page were easy to understand. There was no freeform feedback on this screen.

Final Design The final design ended up adding a few new things. There is a midi playback, that's part of the ABCJS[2] package, and creates a playback of what the exercise sounds like. Considering how easy this feature was to add (It was two lines in the frontend), I thought it would be worth adding in.

The second new feature on this view is the ability to add ABC notation manually. This feature became important for full-stack testing towards the end of creating this project. The music that was being generated was too difficult for me to play, and I wanted to be able to test the same exercise repeatedly after the state of the application had changed.

The third feature was one that I talked about in my preliminary report: the ability to upload audio directly to the frontend. This feature has been extremely nice for testing, but I think users might find it helpful too. For example, recording audio on your phone, and then uploading it to Sight Reader Pro when you get home.

After Recording Exercise Attempt

This screen is extremely similar to the before exercise attempt screen, but with feedback added from the previous attempt. More feedback will be included based on settings selected, including tempo, rhythm accuracy, etc.

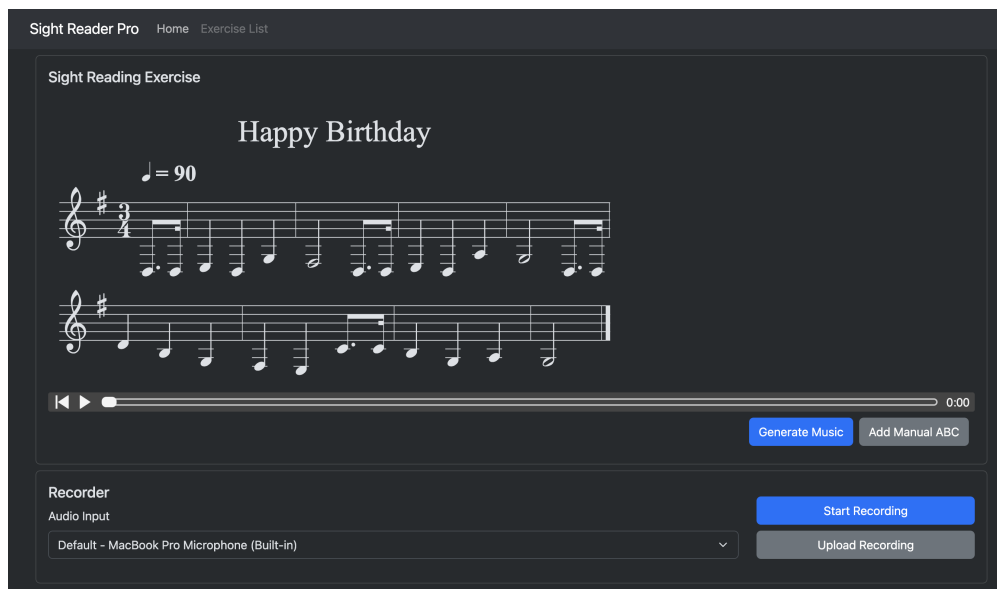


Figure 10: Sight Reading Pro Exercise Before Attempt, Final

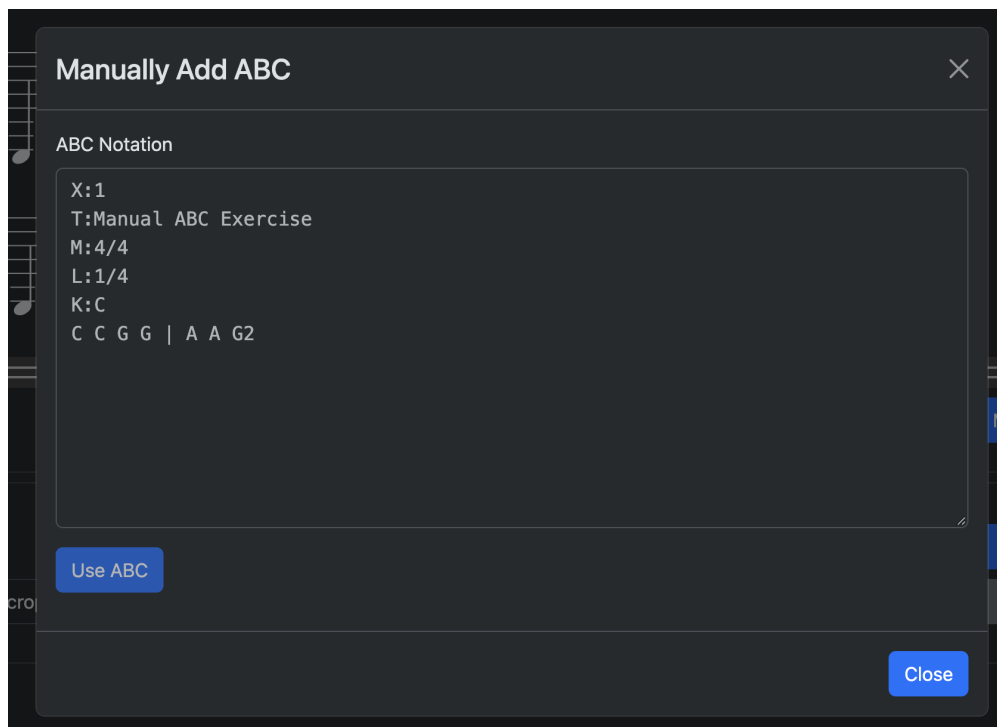


Figure 11: Sight Reading Pro: Add Manual ABC, final

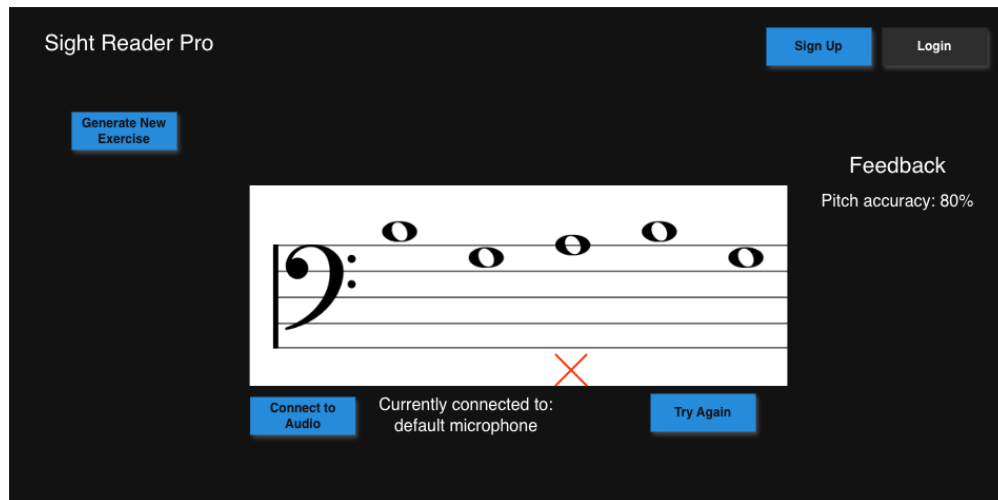


Figure 12: Sight Reading Pro Mockup: Exercise Feedback After Attempt

User Feedback The main feedback I got back from users is that the initial mockup was slightly confusing. They didn't know what the red "X" under the note signified. Users suggested turning the whole note red, and updating the button heirarchy was clear.

This is the feedback from a sight-reading attempt. It shows whether each note was played correctly or not. This sight-reading feedback easy to understand

7 responses

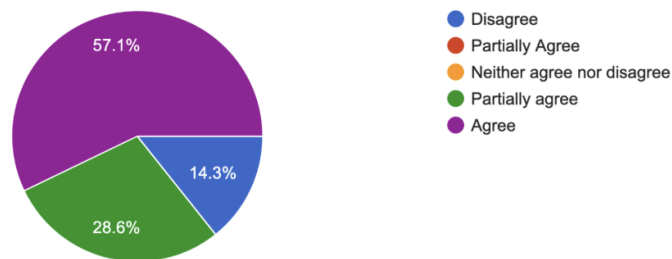


Figure 13: Initial Design Survey: Exercise Feedback

Final Design The final design took the user feedback to heart, and implemented both colored note highlighting for the notes the user missed, and updated buttons that are color coordinated.

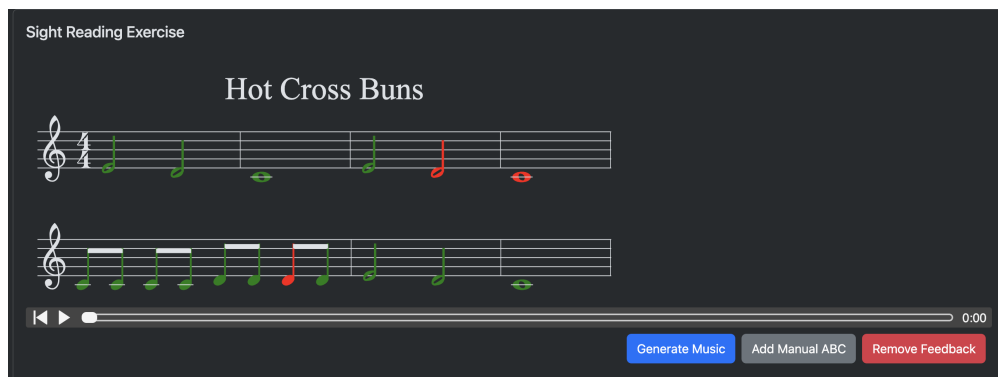


Figure 14: Sight Reading Pro: Exercise Attempt Feedback, Final

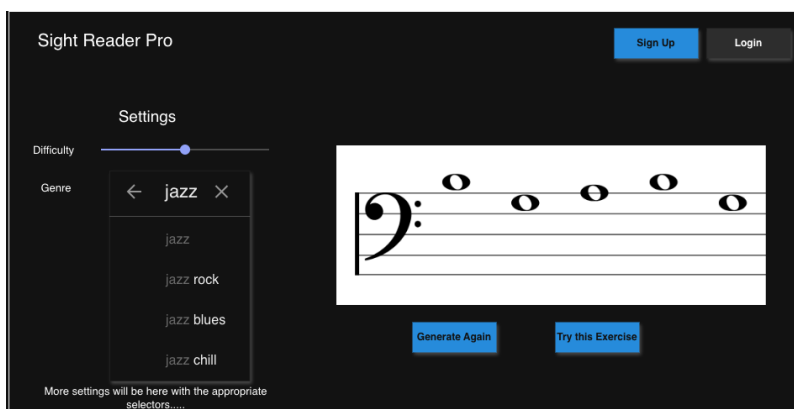


Figure 15: Sight Reading Pro Mockup: New Exercise

Generate New Exercises

In the mockup I have two settings to choose from, a difficulty slider and a genre selector. I envisioned many more settings, but only ended up adding support to select from a list of common instruments.

User Feedback The majority of the users agreed that the initial design gave them all the information they need to select an exercise. There was no freeform feedback.

This is the exercise list view. It shows all of the exercises that you have generated. This screen gives users all the information they need to select an exercise

7 responses

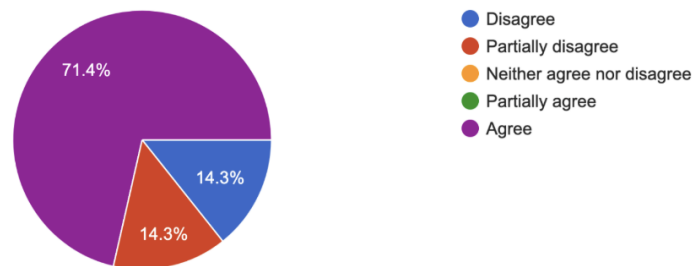


Figure 16: Initial Design Survey: Exercise Feedback

Final Design Since the users liked the initial form, I didn't change it much. I moved the difficulty slider to a dropdown, as prompting with a difficulty number is challenging, and added the option to select an instrument.

Design Choices

Keep it simple

Considering the complexity that's required to generate sight-reading music, and then analyze it, I wanted to keep the frontend web application as simple as possible. There is some customizability, but the core features are simple and intuitive.

Good experience without needing a login

My main goal is to reduce friction in users trying out Sight Reader Pro. All of the core functionality will be in the exercise page, which requires no login. Any person that stumbles across the app can use it right away.

A dark-themed modal window titled "Generate Exercise" with a close button (X) in the top right corner. The window contains three dropdown menus: "Difficulty" set to "Beginner", "Instrument" set to "Bass", and "Genre" set to "Rock". Below these is a text area labeled "Additional Instructions" with the placeholder text "What else would you like to see in this exercise?". At the bottom left is a blue button labeled "Generate Music", and at the bottom right is a blue button labeled "Close".

Figure 17: Sight Reading Pro Mockup: New Exercise

Customizable Exercise Settings

The settings to generate sight-reading exercises will allow users to generate exercises that are specific to them. For example, I only would want to see bass exercises that have a jazz theme. Each user will be able to “choose their own adventure” for what kind of sight-reading experience they want.

Implementation (1653/2500 words)

Docker

The whole project is wrapped in docker[5]. The frontend, the main API, all of the models, and MLFlow[13] are all in their own containers.

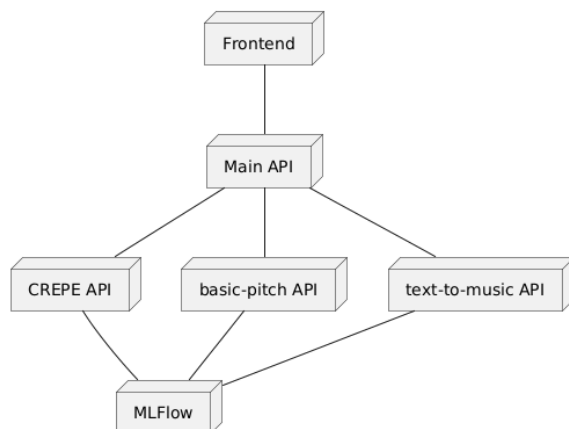


Figure 18: Docker Architecture

There are two defined networks: **frontend-network** and **model-network**. **frontend-network** has the frontend container and the Main API container. The goal of this network was to isolate what could talk to the Main API, and the Frontend. I didn't want any of the model containers being able to communicate to the frontend, and I didn't want the frontend or any external requests to be able to communicate with the model containers, or the MLFlow container.

Using docker allowed me to better control the dependencies for each model. For example, the **text-to-music model** uses python 3.12, whereas **basic-pitch model** uses python 3.10. These projects would not be able to run in the same container, as they both require different versions of Numpy[15]. It also allowed me to experiment with different libraries for analysis, such as music21 and librosa. Being able to destroy a container that didn't work out and spin a new one up was amazing.

Docker Compose

In the main /project directory in my github repository[55], there's my docker-compose. This file controls almost everything about the backend projects, and how they interact with each other.

This is how **basic-pitch** is configured in the **docker-compose.yml** file:

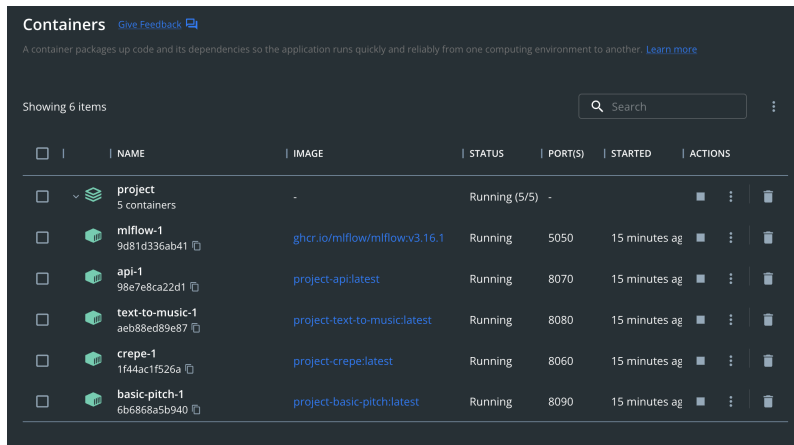


Figure 19: Docker Desktop Capture

```
basic-pitch:
  build:
    context: ./backend/basic-pitch
  networks:
    - model-network
  ports:
    - "8090:8090"
  env_file:
    - .env
```

`basic-pitch` runs on my local port 8090, and I'm able to pass in secrets and environmental variables into the container without having to declare them over and over, as I use them in all the containers.

Example

Lets say I just made a change in `api` and `basic-pitch`. I can rebuild only those containers and spin them up with just two commands:

```
docker compose build api basic-pitch
docker compose up api basic-pitch crepe
```

Run all projects

To run all of the containers in the `docker-compose.yml` file, simply run this from inside of the `/project` directory:

```
docker compose up
```

Frontend

Libraries Used

The frontend was written using React[19] and uses Vite[33] for the docker/production build. I was already familiar with React, as I use it at my day job, and thought it would be the easiest way for me to make the frontend quickly.. The main styling and core components were built using React Bootstrap[20]. I used abcjs[2] for rendering abc notation, and for creating the MIDI playback for the displayed exercise.

structure

- api/
- components/
- pages/
- public/
- App.tsx
- index.tsx

api contains methods for calling the main **api** project. It forces the exact request types and responses that will be coming from each endpoint.

components contains all of the components that make up the application. Examples include:

- **<Navigation />**: The navigation bar at the top of the page.
- **<Recorder />**: Handles all of the audio input into the app. Select an audio device, upload audio, and the button to analyze the audio
- **<Music />**: Renders ABC Notation, renders feedback from the analyze function, and has the generated audio playback functionality
- **<MusicContext />**: Stores the current abc string, and feedback array. Is used in other components that need to access or modify abc/feedback.
- *Modals* Both the *Manual ABC Creation Modal* and the *Generate Music Modal* popups
- *Forms* The forms that are displayed in the modals

pages There's only one file in here, and that's **home.tsx**. This file holds all of the core components needed to render the page.

public Contains only **vmsg.wasm**, the Web Assembly[35] binary required for the **vmsg** package to work. **vmsg** ensures that the recordings sent to the backend are in **.mp3 format**, an issue that caused me many hours of lost time!!

app.tsx Hosts the main wrapper for the application, including the **<Navigation/ >**, and the **<Music />**

index.tsx / index.html The default landing page, and the entry point into the whole application.

other The `package.json` and `package-lock.json` are in `src`. There are also a few helper files, such as `eslint.config.ts`, which ensures formatting and linting happens each time I save a file. There's also vite testing configuration files, that currently do *Nothing!* as I didn't get to any frontend unit testing.

I decided to use TypeScript[32] so that all of the components could have strong typing. This makes it much easier when using shared methods, as the TypeScript compiler will freak out if anything is left untyped.

This app has the same structure as the other docker files, but it doesn't currently work in the docker container. It's on my list of future TODOs!

Main Backend API

This is the API that communicates with the frontend, and with all of the model containers. The backend api also handles the main analysis function for

dependencies

The backend API has a few packages that are different from the model container, including `httpx` to make async requests to the other models, music analysis and utility libraries `librosa`, `pretty_midi`, `mir_eval`, `muspy` and `music21`.

Routers

There are routers for each of the model containers:

- `basic_pitch.py`
- `crepe.py`
- `music_to_text.py`
- `utilities.py`
 - Used to create visualizations
 - As of writing this report, it has the method to create an image from the MIDI file returned from `basic-pitch`

All of the routes use `httpx` to make requests to the model APIs. The majority of them are simple build request, wait for response, return response kind of files.

main.py

Contains the main logic to startup a `FastAPI`[9] server, and add all of the routers to it.

It has two routes: a `hello world` route, and the `analyze` route. The `analyze` route handles processing the user's submitted audio file, and sends it to the `analyze` component.

Also has all wildcard CORS policy, which is not great, but it allows me to debug locally efficiently! This needs to be removed if this app ever gets deployed.

Components

abc_utils.py A few helper functions to help validate and parse the ABC Notation strings that are returned from `music-to-text`.

The `ensure_default_note_length` method adds a normally missing property back into ABC strings. The `abc_to_midi` method uses `music21` to generate a baseline MIDI file to be used to compare against what `basic_pitch` creates. I'm not happy with this method, and it's on my list of things to fix.

analyze.py Contains `analyze_v1`, which currently uses `mir_eval` to match notes in the *baseline MIDI* and the *basic-pitch generated midi*.

Results from crepe are passed in to `mir_eval` for finding the estimated tempo, and to compare against the *baseline MIDI*.

The results are then parsed, and a feedback object and some metrics are returned to the frontend for the user to see.

`analyze_v1` works just okay. It doesn't do well with complex rhythm, or fast, muddy recordings. I plan on implementing an `analyze_v2` with another attempt.

Model APIs

All of the model containers follow the same basic flow, using a version of python that meets the main package requirements, FastAPI for calling the model, and MLFlow for logging each experiment.

Each of the model containers has its own python runtime, its own set of dependencies,

Basic-Pitch API

The base container The basic-pitch documentation recommends `python 10` for running the model. The basic-pitch model gets downloaded directly into the container using this line:

```
RUN apt-get update && apt-get install -y curl && \
    curl -L https://github.com/spotify/basic-pitch/raw/refs/heads/main/
    basic_pitch/saved_models/icassp_2022/nmp.onnx -o ./nmp.onnx
```

I chose to use the onnx model, as it was the model that required the least amount of dependencies.

The API Aside from the hello world endpoint, there's the `/midi` endpoint which does all logic for running basic pitch. `/midi` expects the audio file that it's going to convert.

Before any conversion begins, an *MLFlow* experiment is started, and logs all of the parameters.

Text-To-Music API

The base container This container needed quite a few special packages to make the *text-to-music* model work:

- `libffi-dev`
- `package-config`
- `libssl-dev`

It also installs `rust`[22], which is needed for `torch`[18] to work.

The API Only one endpoint aside from a hello world, which is `/generate`. It takes in the prompt that was generated on the frontend (based on what the user put in the generate exercise form), and runs inference on the `text-to-music` model.

MLFlow experiment tracking is enabled for each inference run, so I can keep track of my different hyperparameters I've used on this model.

text-to-music Inference Based on the *text-to-music* model card[24], the `samplings` library is used to sample `top_p_sampling` and `temperature_sampling`. I attempted to use a normal HuggingFace `transformers`[58] `generate` method, but no valid ABC notation came out of it.

For now, this weird inference setup stays. I want to try and make it more efficient.

CREPE API

The base container *CREPE* demanded a very specific version of setup tools in order to compile. I added a `pip install setuptools<81` command inside of the docker container before the normal `requirements.txt` were installed.

CREPE uses `Tensorflow`[29] to load the model and run inference.

The API There is one main method, `/pitch-tracker` that calls *CREPE* with the default parameters that were in the example starter code, because the defaults work great! It then returns the predictions that *CREPE* generates.

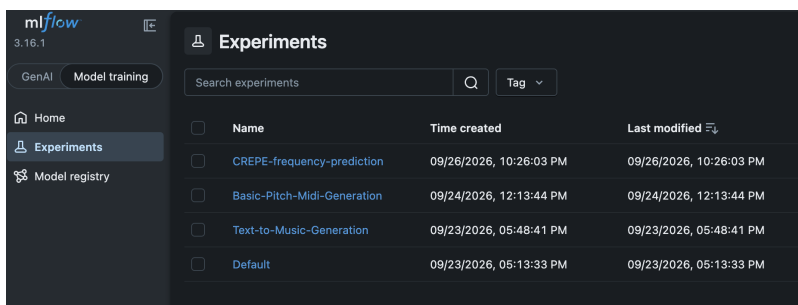
There's also `/pitch-tracker-image`, which produces an activation plot based on the predictions. The code that generates the image is from the *crepe* repository,

because it was not included in the CREPE `pip` package. This is only used for testing purposes, and is not callable from the frontend.

MLFlow

MLFlow[13] is an experiment tracker for machine learning models. It can track hyperparameters, and metrics about the container it's running in.

I use the results from MLFlow container in the *Evaluation* part of this report.



☐	Name	Time created	Last modified
☐	CREPE-frequency-prediction	09/26/2026, 10:26:03 PM	09/26/2026, 10:26:03 PM
☐	Basic-Pitch-Midi-Generation	09/24/2026, 12:13:44 PM	09/24/2026, 12:13:44 PM
☐	Text-to-Music-Generation	09/23/2026, 05:48:41 PM	09/23/2026, 05:48:41 PM
☐	Default	09/23/2026, 05:13:33 PM	09/23/2026, 05:13:33 PM

Figure 20: MLFlow screenshot of experiments

At a high level

There are two main actions that a user of Sight Reader Pro does:

- Generate sight-reading exercises
- Analyze sight-reading exercises

The following picture shows the major API calls that happen between the containers to create exercises and analyze audio:

Evaluation (721/2500 words)

Model Evaluation

One of the core parts of choosing this project template was model evaluation. My main goals for choosing models were the following:

- Does this model have fast inference?
- Can the model generate a variety of music styles?
- Output ABC consistently

Text2Midi

notebook for my exploration is at `project/model_exploration/text2midi.ipynb`

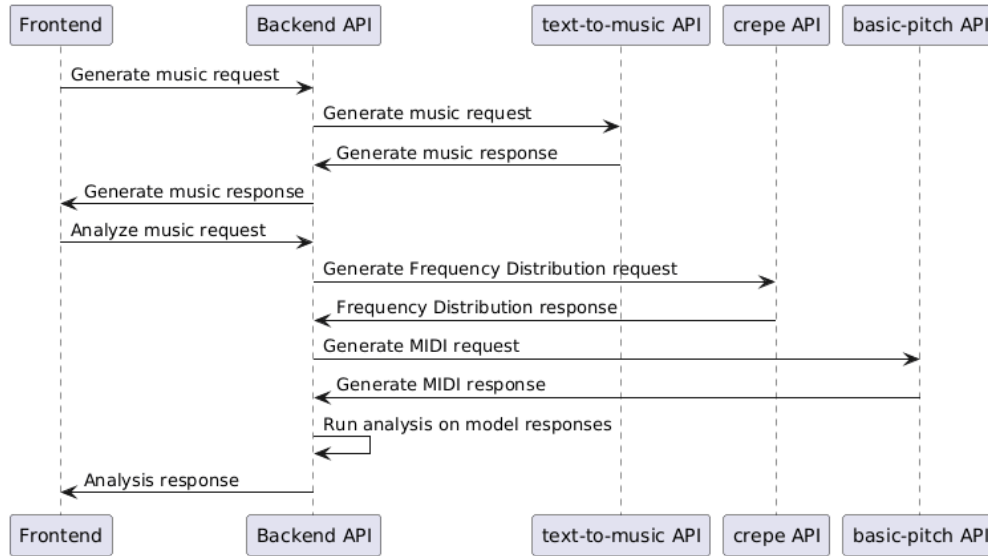


Figure 21: Core Functionality API Calls

This is the only text-to-midi model that I tried. It took almost 2 hours to run inference in a google colab space. There were some issues with the example code that I tried causing an error when trying to run the decoder and create the midi file.

The inference time and the amount of compute made me say no to this model.

Text-To-Music

text-to music consistently generates readable **abc** notation. The only time it doesn't is when I make the **max_iter** parameter too low. I really like the variety of music this model makes. I feel like I'm playing roulette with this model, never quite knowing what I'll get. This leads to a variety of strange things, such as generating exercises with lots of rests, or exercises that are extremely difficult.

System Metrics During inference, the memory use and CPU use spike dramatically. I believe it would be worth exploring what's taking up so many resources, and if the inference loop I'm currently using can be optimized.

MLFlow data All of the **text-to-data** inference runs were logged to MLFlow. Not all of them succeeded. There were a lot of times where inference took a lot longer for one prompt than it did for another. The **text-to-music: max_length vs Duration** plot visualizes this. It might be worth looking into why sometimes inference takes so much longer.



Figure 22: text-to-music Generates lots of rests

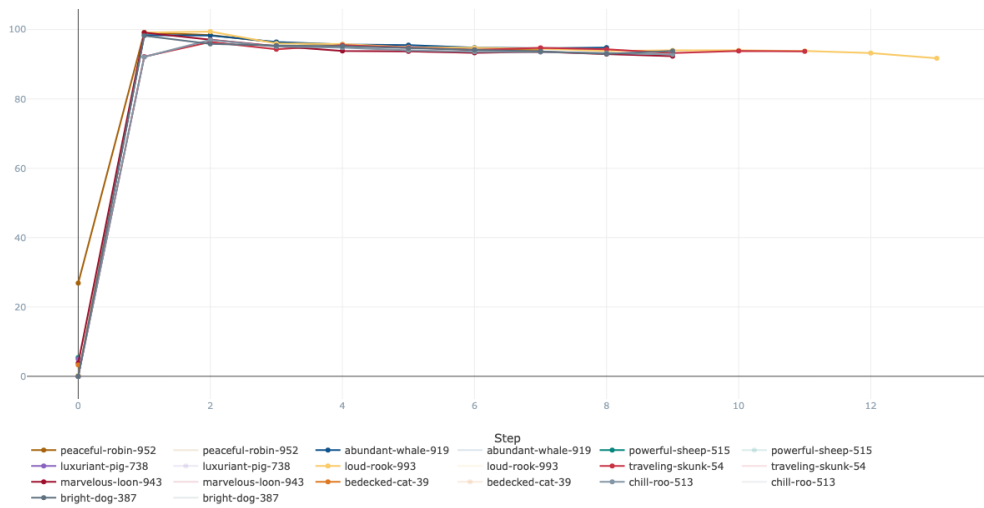


Figure 23: text-to-music System CPU Utilization

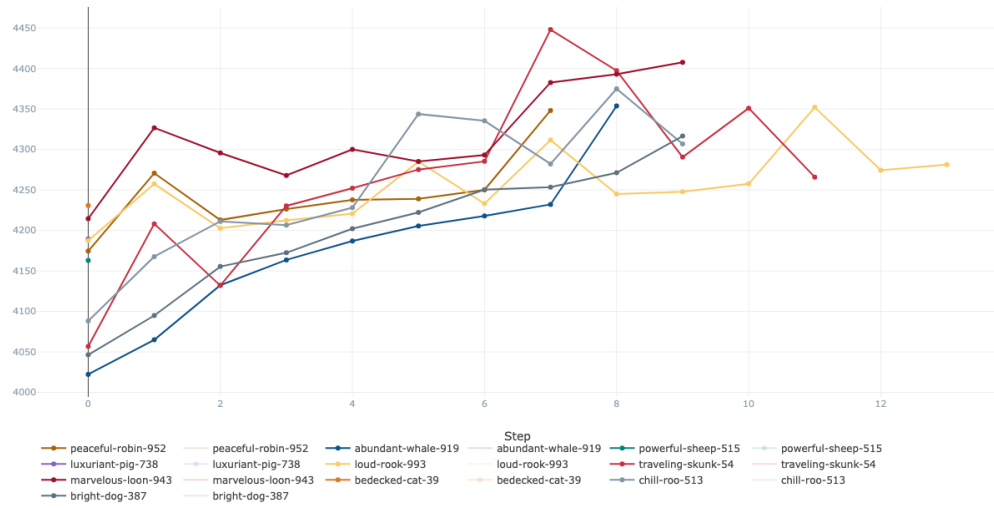


Figure 24: text-to-music System Memory Utilization

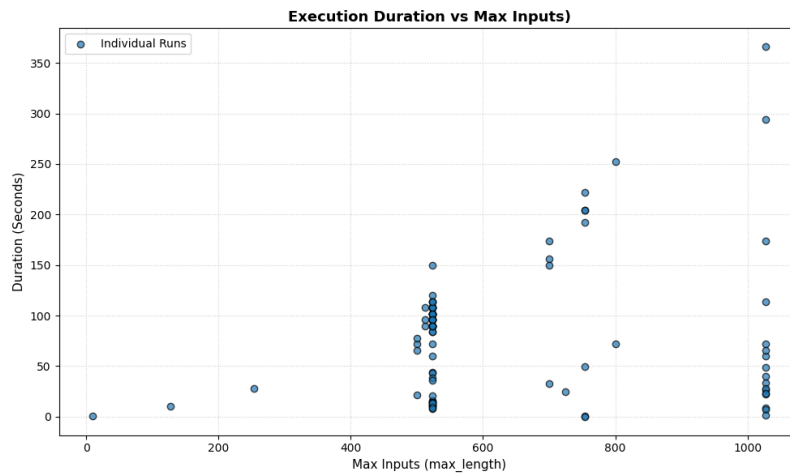


Figure 25: text-to-music: max_length vs Duration plot

I also thought that the size of `max_input` was introducing failures at lower lower numbers of iterations. While it's true that there were some errors,

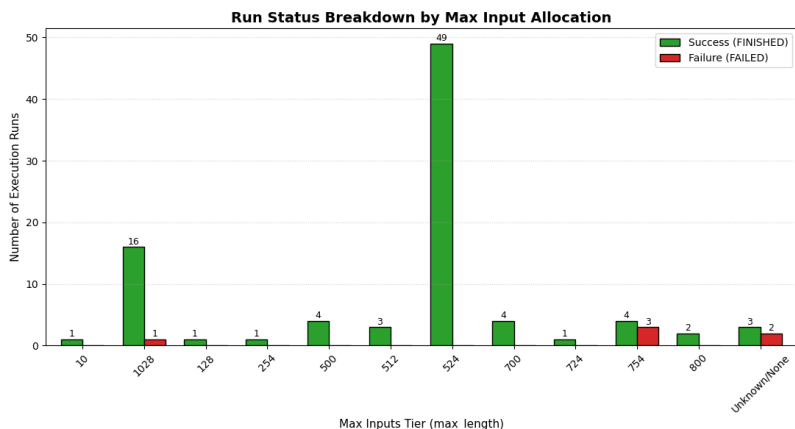


Figure 26: text-to-music: `max_length`, iterations & errors

basic-pitch

Basic pitch turned out to be a fast and efficient way to generate midi files. I need to spend more time with the hyper parameters to play with note grouping, and the minimum and maximum frequency. For example, a bass would never go above 300hz, so it should ignore any noises in that range. Same with trying to figure out what the tempo of the upload is before running basic-pitch, so that the groupings are more informed during analysis.

Failed Approaches

Porting TextToMusic to ONNX Runtime

Earlier on in this project, I was playing around with the idea of using Transformers.js[10] to grab all of the models for this project, and have inference done using ONNX Runtime[16]. This would allow me to run model inference on the client side, which would be so cool! The Transformers.js documentation made it seem like it would be easy enough, so I gave it a shot. The main python notebook (located at `project/model-exploration/text_to_music_onnx_conversion.ipynb` in the project repo[55]) use to convert Text-To-Music to an onnx format seemed like it was going well, just running into typical input/output shape issues. I made it as far as uploading it to HuggingFace[54] before I realized running inference on this model required a custom python library, *Samplings*[23]. For a few hours I tried to re-implement both the *TopPSampling* and *TemperatureSampling* methods before I decided to just use what the original author of the model had used, and go back to doing a full python implementation.

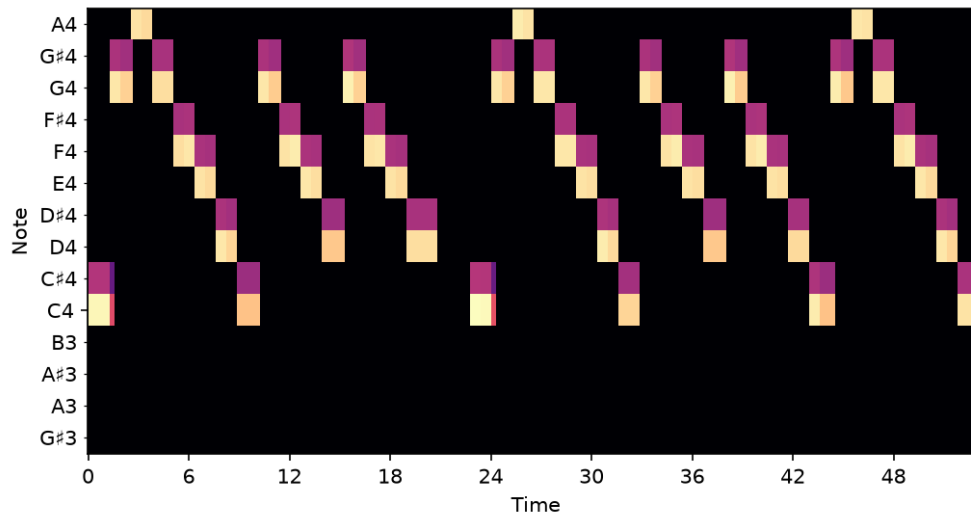


Figure 27: basic-pitch: Twinkle Twinkle Little Star midi visualization

Limitations

Generated Music

Even when genre, difficulty and instrument are provided in the prompt, `text-to-music` seems to ignore it and generate whatever it wants. There are also issues with random rests being generated. I believe the prompt and hyper-parameters need to be re-evaluated to improve the results.

Evaluation set

I kept using the same two or three `.wav` files that I recorded myself for evaluating the functions. I believe that there are many bugs because of this.

In the future, having well defined evals ahead of time will make it easier to test my application, and I'll get a better idea as to how a variety of different styles of music, and inputs perform in the system.

Conclusion (375/1000 words)

If I were to do it again...

Using Music Analysis Libraries First

If I were to start this project over, I believe my approach would change considerably. I think I probably would've stuck with using something like `gradio`[38] or `streamlit`[28] to quickly make the frontend would've cut down my workload for

getting a testable user interface, and allow me to put more time into prompt tuning the models I chose, and maybe even fine tuning models that would work better to generate sight-reading passages.

I also think that finding smaller, more well established baselines for these tasks would be helpful. I chose the models for the application before I stumbled upon `librosa`, `mir_eval` and `muspy`. I believe if I were to implement a processing pipeline with just these libraries, I could get 90% of the functionality with 10% of the headache I endured completing this project.

Move away from FastAPI

I think my use of FASTAPI in this project was overkill. It was nice to have all of the models in their own containers, but I think

Further Work

Rethink the analyze function

I know that there are better ways to use the *CREPE* and *basic-pitch*

Clean Up Front End

Component Clean Up The components folder needs love. Move all of the components into their own folders, and have the styles, and similar components live together. Having a wall of components staring at me each time I look at the file tree really sucks.

Better State Management Take a look at this code that's currently in `components/recorder.tsx`:

```
const [devices, setDevices] = useState<MediaDeviceInfo[]>([]);
const [selectedDevice, setSelectedDevice] = useState<string>("");
const [recording, setRecording] = useState(false);
const [recordingBlob, setRecordingBlob] = useState<Blob | null>(null);
const [audioUrl, setAudioUrl] = useState<string | null>(null);
const [hasRecording, setHasRecording] = useState(false);
const [isLoading, setIsLoading] = useState(false);
```

What are we doing here. Choose a better state manager. Don't do this. This has become really hard to work with, and is the source of some gnarly bugs, such as `.mp3` files being invalid.

I think that moving over to Redux[21], or another state management solution would do worlds for the frontend.

Explore prompt optimization using DS.py

Right now all of my prompting to the `text-to-music` model are created manually. I'm doing my best to follow the prompt style of the examples, but the results really seem to go all over the place.

Using something like `DS.py`[6] to program more efficient prompts seems like an interesting approach to get more out of `text-to-music` without having to finetune it.

Acknowledgements

- I did my best highlighting each section of this PDF for my word counts in the headers. My apologies if the counts are over or under inflated.
- I'm grateful to my sister Anna, and my partner J for proof reading through this report too many times to count, and supporting me during stressful times.
- Pandoc was used to generate this report from a markdown file
- Zotero was used to manage sources, and generate a BibTex file for references/citations
- draw.io was used for all of the Sight Reader Pro mockups
- <https://plantuml.com/> was used for creating my architecture diagrams

References

- [1] Abc notation. <https://abcnotation.com/>.
- [2] Abcjs: Javascript Music Library. <https://abcjs.net>.
- [3] CREPE Interactive Demo. <https://marl.github.io/crepe/>.
- [4] Crepe: CREPE pitch tracker.
- [5] Docker | Secure Sandboxes for AI Agents.
- [6] DSPy. <https://dspai.current/>.
- [7] 2024. Empowering Students through Sight Reading. <https://nafme.org/>.
- [8] 2023. Facebook/audiogen-medium · Hugging Face. <https://huggingface.co/facebook/audiogen-medium>.
- [9] FastAPI - FastAPI. <https://fastapi.tiangolo.com/>.
- [10] 2026. Huggingface/transformers.js. Hugging Face.
- [11] 2026. MIDI. *Wikipedia*. (Sep. 2026).
- [12] Mir_eval Documentation — mir_eval 0.8.2 documentation. <https://mir-eval.readthedocs.io/latest/index.html>.
- [13] MLflow - Open Source AI Platform for Agents, LLMs & Models. *MLflow AI Platform*. <https://mlflow.org>.
- [14] Muspy — MusPy documentation. <https://hermandong.com/muspy/doc/muspy.html>.
- [15] NumPy. <https://numpy.org/>.
- [16] ONNX Runtime. <https://onnxruntime.ai/>.
- [17] Onnx-community/Qwen3-0.6B-ONNX · Hugging Face. <https://huggingface.co/onnx-community/Qwen3-0.6B-ONNX>.
- [18] PyTorch. *PyTorch*.
- [19] React. <https://react.dev/>.
- [20] React Bootstrap | React Bootstrap. <https://react-bootstrap.netlify.app/>.
- [21] Redux - A JS library for predictable and maintainable global state management | Redux. <https://redux.js.org/>.
- [22] Rust Programming Language. <https://rust-lang.org/>.
- [23] 2022. Samplings. *PyPI*. <https://pypi.org/project/samplings/>.
- [24] 2022. Sander-wood/text-to-music · Hugging Face. <https://huggingface.co/sander-wood/text-to-music>.

- [25] 2026. Sander-wood/wikimusictext · Datasets at Hugging Face. <https://huggingface.co/datasets/sander-wood/wikimusictext>.
- [26] Sight Reading Factory®. <https://www.sightreadingfactory.com>.
- [27] Sight Reading Trainer. <https://sightreading.training/>.
- [28] 2021. Streamlit • A faster way to build and share data apps. <https://streamlit.io/>.
- [29] TensorFlow. <https://www.tensorflow.org/>.
- [30] TensorFlow.js | Machine Learning for JavaScript Developers. *TensorFlow*. <https://www.tensorflow.org/js>.
- [31] Text2midi: Generating Symbolic Music from Captions. <https://arxiv.org/html/2412.16526v1>.
- [32] TypeScript Website. <https://www.typescriptlang.org/>.
- [33] Vite. *vitejs*. <https://vite.dev>.
- [34] 2026. Web MIDI API | MDN. *MDN Web Docs*. https://developer.mozilla.org/en-US/docs/Web/API/Web_MIDI_API.
- [35] WebAssembly. <https://webassembly.org/>.
- [36] What is Music21? — Music21 Documentation. <https://music21.org/music21docs/about/what.html>.
- [37] 2025. Wikimedia/wikipedia · Datasets at Hugging Face. <https://huggingface.co/datasets/wikimedia/wikipedia>.
- [38] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. and Zou, J. 2019. Gradio: Hassle-free sharing and testing of ML models in the wild.
- [39] Bittner, R.M., Bosch, J.J., Rubinstein, D., Meseguer-Brocal, G. and Ewert, S. 2022. A Lightweight Instrument-Agnostic Model for Polyphonic Note Transcription and Multipitch Estimation. *arXiv*.
- [40] Carnovalini, F., Espírito Santo, L. and Wiggins, G.A. 2025. Personalized Music Education: A Systematic Review of AI Generation Methods. *IEEE Access*. 13, (2025), 207433–207447. <https://doi.org/10.1109/ACCESS.2025.3639898>.
- [41] Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y. and Défossez, A. 2024. Simple and Controllable Music Generation. *arXiv*.
- [42] Donahue, C., Thickstun, J. and Liang, P. 2022. Melody transcription via generative pre-training. *arXiv.org*. <https://arxiv.org/abs/2212.01884v1>.
- [43] Dong, H.-W., Chen, K., McAuley, J. and Berg-Kirkpatrick, T. 2020. MusPy: A Toolkit for Symbolic Music Generation. *arXiv*.
- [44] Grattafiori, A. et al. 2024. The Llama 3 Herd of Models. *arXiv*.
- [45] Hayward, C.M. and Gromko, J.E. 2009. Relationships among Music Sight-Reading and Technical Proficiency, Spatial Visualization, and Aural Discrimination. *Journal of Research in Music Education*. 57, 1 (2009), 26–36.
- [46] Kim, J.W., Salamon, J., Li, P. and Bello, J.P. 2018. CREPE: A Convolutional Representation for Pitch Estimation. *arXiv*.

- [47] Kreuk, F., Synnaeve, G., Polyak, A., Singer, U., Défossez, A., Copet, J., Parikh, D., Taigman, Y. and Adi, Y. 2023. AudioGen: Textually Guided Audio Generation. arXiv.
- [48] Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V. and Zettlemoyer, L. 2019. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. arXiv.
- [49] Mauch, M. and Dixon, S. 2014. PYIN: A fundamental frequency estimator using probabilistic threshold distributions. *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (Florence, Italy, May 2014), 659–663.
- [50] McFee, B., Raffel, C., Liang, D., Ellis, D.P.W., McVicar, M., Battenberg, E. and Nieto, O. 2015. Librosa: Audio and Music Signal Analysis in Python. *SciPy 2015*. (Jul. 2015). <https://doi.org/10.25080/Majora-7b98e3ed-003>.
- [51] OpenAI et al. 2024. GPT-4 Technical Report. arXiv.
- [52] Raffel, C., McFee, B., Humphrey, E.J., Salamon, J., Nieto, O., Liang, D. and Ellis, D.P.W. A TRANSPARENT IMPLEMENTATION OF COMMON MIR METRICS.
- [53] Schreiber, H. and Müller, M. 2018. A Single-step Approach to Musical Tempo Estimation using a Convolutional Neural Network. *Proceedings of the 19th International Society for Music Information Retrieval Conference*. (Sep. 2018), 98–105. <https://doi.org/10.5281/zenodo.1492353>.
- [54] Thoraldson, T. Tthoraldson/text-to-music-onnxruntime · Hugging Face. <https://huggingface.co/tthoraldson/text-to-music-onnxruntime>.
- [55] Thoraldson, T. 2026. Tthoraldson/uol-final-project.
- [56] Touvron, H. et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv.
- [57] Wang, Y., Wu, S., Hu, J., Du, X., Peng, Y., Huang, Y., Fan, S., Li, X., Yu, F. and Sun, M. 2025. NotaGen: Advancing Musicality in Symbolic Music Generation with Large Language Model Training Paradigms. arXiv.
- [58] Wolf, T. et al. 2020. Transformers: State-of-the-Art Natural Language Processing. Association for Computational Linguistics.
- [59] Wu, S., Li, X., Yu, F. and Sun, M. TunesFormer: Forming Irish Tunes with Control Codes by Bar Patching.
- [60] Wu, S.-L., Kim, Y. and Huang, C.-Z.A. 2025. MIDI-LLM: Adapting Large Language Models for Text-to-MIDI Music Generation. arXiv.
- [61] Wu, S., Wang, Y., Guo, R.Y.Z., Tan, X., Zhang, G., Zhou, M., Chen, J., Mu, X., Gao, Y., Dong, Y., Liu, J., Li, X., Yu, F. and Sun, M. CLaMP 2: Multimodal Music Information Retrieval Across 101 Languages Using Large Language Models.
- [62] Yang, A. et al. 2025. Qwen3 Technical Report. arXiv.
- [63] Yuan, R. et al. 2024. ChatMusician: Understanding and Generating Music Intrinsically with LLM. arXiv.